Laboratory Report: Static Code Analysis

Course: Software Engineering Laboratory (Lab 5)

Student Name: Aarav Yuval B G

USN: PES1UG24AM004

Assigned Repository: https://github.com/SETAPESU26/L5_05_food_delivery.git

Working Branch: `lab5-pes1ug24am004-static-analysis`

Date: 2026-10-23

---

1. Objective

The objective of this laboratory exercise is to apply static code analysis tools (Pylint, Bandit, and Flake8) to identify, analyze, and resolve code quality issues, security vulnerabilities, styling flaws, and logical defects in a Python-based Food Delivery Order Queue Manager (`delivery_system.py`). The exercise demonstrates establishing verifiable baseline metrics, executing targeted refactoring across three distinct Git commits, implementing automated verification tests, and achieving clean, production-grade static analysis scores.

---

2. Overview of the Application

The target application, `delivery_system.py`, manages an order queue for a commercial food delivery kitchen:

- **Menu Loading:** Reads dish names and pricing configurations from a JSON file (`menu.json`), falling back to built-in default prices if the configuration file is missing.
- **Order Queue Management:** Creates new customer orders, computes item totals, tracks order history, and manages sequential order identifiers.
- **Status Progression:** Updates orders through valid kitchen and dispatch states (`placed` -> `cooking` -> `out_for_delivery` -> `delivered`).
- **Revenue Accounting:** Aggregates and calculates daily earned revenue by summing totals from delivered orders.
- **Persistence Layer:** Serializes queue state to disk for backup and restores queue state on recovery.
- **Diagnostics:** Prints timestamped summaries of orders, customer details, delivery statuses, and amounts.

---

3. Principles of Static Code Analysis and Tool Specializations

Static code analysis evaluates source code without executing the program. By parsing source files into an Abstract Syntax Tree (AST) or token streams, static analyzers identify syntactical errors, dangerous idioms, security vulnerabilities, and stylistic non-conformance prior to deployment or testing.

Three specialized tools were applied:

Pylint (Semantic and Code Quality Analyzer):

- Analyzes code structure, logical inconsistencies, variable scopes, exception handling practices, and API documentation.
- Evaluates adherence to software design standards and provides a composite quality rating out of 10.0.

Bandit (Security Vulnerability Scanner):

- Inspects the AST for security-sensitive coding patterns cataloged in the Common Weakness Enumeration (CWE) database.
- Flags dangerous modules, insecure deserialization, command injection hazards, and suppressed exception blocks.

Flake8 (Style and Complexity Linter):

- Combines PyFlakes, PyCodeStyle, and McCabe complexity analysis.
- Enforces strict PEP 8 compliance, formatting limits (e.g., maximum line length of 79 characters), and identifies undeclared or unused symbols.

---

4. Baseline Findings and Technical Risks

Prior to any code changes, baseline reports were generated and preserved in `reports/before/`:

Tool	Baseline Metric	Primary Violations	Technical Risk
Pylint	6.27 / 10	`W0102` (mutable default `[]`), `W0702` (bare except), `W0718` (broad exception), `W1514` (unspecified encoding), `R1732` (unclosed file handles), `C0103` (naming conventions), `C0114`/`C0116` (missing docstrings).	Cross-call state contamination via shared list; resource leaks from unclosed file descriptors; suppressed errors causing undetectable failures.
Bandit	3 Issues (1 Medium, 2 Low)	`B403` (`import pickle`), `B301` (`pickle.load`), `B110` (`try...except: pass`).	Arbitrary code execution vulnerability via untrusted pickle deserialization (CWE-502); silent suppression of critical system failures (CWE-703).
Flake8	12 Violations	`E722` (bare except), `E501` (lines > 79 characters), `E302` (insufficient blank lines before functions), `E305` (insufficient blank lines after functions).	Degraded code readability, non-standard code layout, and risky exception handling.
Manual Review	Logic Defects	Status updates allowed arbitrary states (e.g., `"teleported"`); empty orders allowed; unknown menu items silently charged 99.	Order state corruption, invalid financial billing, and unauthorized order generation.

Detailed raw baseline logs are available in:

- [Pylint Baseline Report](file:///C:/Users/Aarav/Documents/Studies/Sem5/Lab/SE/food_delivery/reports/before/pylint_report.txt)
- [Bandit Baseline Report](file:///C:/Users/Aarav/Documents/Studies/Sem5/Lab/SE/food_delivery/reports/before/bandit_report.txt)
- [Flake8 Baseline Report](file:///C:/Users/Aarav/Documents/Studies/Sem5/Lab/SE/food_delivery/reports/before/flake8_report.txt)

---

5. Remediations Implemented

Refactoring was structured into discrete logical phases:

Dangerous Default Parameter Resolution:

- Parameter `order_history=[]` in `create_order` was replaced with `order_history=None`. Inside the function, the parameter is initialized via `if order_history is None: order_history = []`, guaranteeing an independent list for every function invocation.

Safe Serialization Migration:

- Completely removed `import pickle`, `pickle.dump`, and `pickle.load`. Replaced with `json.dump` and `json.load` in `backup_orders` and `restore_orders`. Added an integer-conversion loop during restoration (`orders[int(order_id)] = order_data`) so dictionary indexing remains uniform.

Exception Handling and Error Pre-conditions:

- Eliminated `try...except: pass` in `update_order_status`. Replaced with deterministic pre-condition validations checking `order_id in orders` and `status in VALID_STATUSES`.
- Replaced broad `except Exception:` in `read_menu_config` with specific `except (FileNotFoundError, json.JSONDecodeError, OSError):`.

Input Validation and Business Logic Hardening:

- Added validation in `create_order` to reject non-string or whitespace-only customer names, empty item lists, and items not present on the menu (`raise ValueError`).

Resource Management:

- Converted all file reading and writing operations to use Python's `with open(..., encoding="utf-8") as f:` context managers, guaranteeing automatic file descriptor cleanup.

Code Style and PEP 8 Formatting:

- Added comprehensive docstrings to module and functions.
- Renamed `printQueue` to `print_queue`.
- Formatted all lines to 79 characters or fewer with standard two-line function separation.
- Wrapped top-level script execution in `if __name__ == "__main__":`.

---

6. Before-and-After Metric Comparison

Analysis Dimension	Baseline State	Final State	Outcome
Pylint Score	6.27 / 10	**10.00 / 10**	Exceeds > 9.5 target (+3.73 improvement)
Bandit Findings	3 (1 Medium, 2 Low)	**0 Issues**	All CWE security issues resolved
Flake8 Violations	12 violations	**0 Violations**	Clean PEP 8 compliance
Pickle Security Hazard	Present (`orders.pkl`)	**Removed**	Replaced with secure JSON format
Input Validation	None (empty/unknown accepted)	**Enforced**	Strict input and status validation
Automated Unit Tests	None (0 tests)	**12 Passed / 12 Ran**	100% test coverage of core logic

Detailed final reports are available at:

- [Pylint Final Report](file:///C:/Users/Aarav/Documents/Studies/Sem5/Lab/SE/food_delivery/reports/after/pylint_report.txt)
- [Bandit Final Report](file:///C:/Users/Aarav/Documents/Studies/Sem5/Lab/SE/food_delivery/reports/after/bandit_report.txt)
- [Flake8 Final Report](file:///C:/Users/Aarav/Documents/Studies/Sem5/Lab/SE/food_delivery/reports/after/flake8_report.txt)

---

7. Automated Test Suite Results

A dedicated automated test suite (`test_delivery_system.py`) was created using Python's `unittest` framework and executed via `pytest`. All 12 test cases executed cleanly in 0.10s with zero failures:

`test_default_menu_loaded`: Confirms fallback menu loading when external file is absent.

`test_read_menu_config_from_valid_file`: Confirms custom menu JSON loading.

`test_create_valid_order`: Validates order placement, ID generation, and total pricing.

`test_create_order_empty_customer_rejected`: Confirms rejection of empty/whitespace customer names.

`test_create_order_empty_items_rejected`: Confirms rejection of empty item selections.

`test_create_order_invalid_item_rejected`: Confirms rejection of unlisted menu items.

`test_create_order_history_independence`: Proves order histories remain isolated between calls.

`test_update_order_status_success`: Confirms valid status progression (`placed` -> `cooking` -> `out_for_delivery` -> `delivered`).

`test_update_order_status_invalid_status_rejected`: Confirms invalid statuses like `"teleported"` are rejected.

`test_update_order_status_nonexistent_order_rejected`: Confirms nonexistent order IDs fail gracefully.

`test_daily_revenue_calculation`: Confirms revenue sums only delivered orders.

`test_safe_backup_and_restore_json`: Confirms loss-free JSON persistence and integer key restoration.

---

8. Git Commit Log

Work was tracked in exactly three logical assignment commits on branch `lab5-pes1ug24am004-static-analysis`:

Commit 1: `Fix Pylint code quality and correctness issues`

- Remediated mutable default arguments, bare exception blocks, broad exceptions, unclosed files, docstring absences, and naming conventions. Raised Pylint score to 10.00/10.

Commit 2: `Resolve Bandit security findings`

- Eliminated `pickle` import and calls. Migrated persistence to safe JSON format. Introduced `reset_system()` and the 12-test automated unit test suite.

Commit 3: `Fix Flake8 style issues and finalize lab deliverables`

- Applied final PEP 8 formatting adjustments, verified 0 Flake8 violations, generated final static analysis reports, created all documentation deliverables (`issues.md`, `report.md`, `reflection.md`, `VIVA_NOTES.md`, `PRESENTATION_SUMMARY.md`), and prepared package deliverables.

---

9. Conclusion and Limitations

Key Takeaways

- Static code analysis provides immediate, automated feedback on code security, maintainability, and standards compliance.
- Replacing unsafe patterns (such as `pickle` with `json`, and bare `except: pass` with explicit precondition checks) prevents high-severity exploits and silent system failures.
- Static analysis and dynamic automated testing must be paired: static tools verify syntax and defensive idioms, while automated tests verify operational business logic.

Limitations

- The project runs an in-memory queue manager suitable for a single kitchen node; scaling to distributed operations would require database transactions, concurrency locking, and token-based authentication.